Design, Kinematic Analysis, and Experimental Validation of a 4-DOF Service Robotic Manipulator for Automated Beverage Preparation

Andrew Abdelmalak
Dept. of Mechatronics Engineering
German University in Cairo (GUC)
Cairo, Egypt

Daniel Boules
Dept. of Mechatronics Engineering
German University in Cairo (GUC)
Cairo, Egypt

David Girgis
Dept. of Mechatronics Engineering
German University in Cairo (GUC)
Cairo, Egypt

Kirolous Kirolous
Dept. of Mechatronics Engineering
German University in Cairo (GUC)
Cairo, Egypt

Samir Yacoub
Dept. of Mechatronics Engineering
German University in Cairo (GUC)
Cairo, Egypt

Youssef Salama
Dept. of Mechatronics Engineering
German University in Cairo (GUC)
Cairo, Egypt

***Abstract*—This paper presents the complete design, kinematic analysis, simulation, and experimental validation of a desktop-scale 4-DOF service robotic manipulator for automated beverage preparation. The manipulator is constructed from 3D-printed PLA components and actuated by three MG996R high-torque servos and three SG90 micro servos. Kinematic modeling follows the Denavit–Hartenberg convention with link lengths $d_1 = 0.04355\,\mathbf{m}$, $a_2 = 0.140\,\mathbf{m}$, $a_3 = 0.133\,\mathbf{m}$, and $a_4 = 0.109\,\mathbf{m}$. A numerical Newton–Raphson inverse kinematics solver achieves sub-millimeter position convergence ($< 10^{-4}\,\mathbf{m}$ in ROS Python using Damped Least-Squares regularization with $\lambda = 0.01$, and $< 10^{-3}\,\mathbf{m}$ in MATLAB using the Moore–Penrose pseudoinverse), while warm-start joint recovery and undamped velocity roundtrip tests achieve machine-precision residuals ($10^{-17}\,\mathbf{rad}$ and $\leq 2.2 \times 10^{-15}\,\mathbf{rad/s}$). Two task-space trajectories—a three-sided square path for pick-and-place operations and a circular path for stirring—are implemented and validated across Gazebo, Simscape Multibody, and physical hardware platforms. A tea-preparation demonstration involving sugar transport, spoon manipulation, and stirring confirms the system's capability for coordinated multi-step service tasks using accessible, educational-grade hardware.**

***Index Terms*—Service robotics, robotic manipulator, Denavit–Hartenberg convention, inverse kinematics, trajectory planning, beverage preparation, 3D printing, Gazebo, Simscape.**

I. Introduction

Service robotics is increasingly transforming customer-oriented industries by automating tasks that demand consistency, hygiene, and precision. Among these applications, beverage preparation has emerged as a key domain where robots can deliver uniform quality while reducing human workload [1]. Recent studies have demonstrated the growing acceptance of robotic baristas and automated food preparation systems, where robots are valued for their ability to operate continuously without fatigue.

Several robotic coffee preparation systems have been proposed. Setiawan and Ma'arif [2] developed a PLC-based coffee-stirring system with PID control, demonstrating how controlled motion improves drink uniformity and prevents spillage. Michalková et al. [3] automated an espresso machine using a FANUC industrial arm and PLC logic, validating high-precision manipulators in café settings. Nurba et al. [4] developed a 3-DOF manipulator applying Denavit–Hartenberg (DH) forward kinematics, achieving repeatability deviations below 0.06 mm. Salman et al. [5] employed the DH convention to model a 3-DOF arm with MATLAB GUI simulation and Arduino-based actuation, while Waluyo et al. [6] demonstrated an Arduino-based robot arm for coffee preparation.

However, most existing systems either target industrial-scale automation or are limited to single-function prototypes. Limited attention has been given to compact, educational-grade manipulators tailored for multi-step service applications using accessible components. This paper addresses this gap through five contributions:

1) Design and fabrication of a 4-DOF desktop manipulator using 3D-printed parts and locally sourced servos.
2) Complete DH-based kinematic modeling including forward kinematics, numerical inverse kinematics, and velocity kinematics with analytical Jacobian derivation.
3) Dual-platform simulation validation in both Gazebo (ROS-integrated) and MATLAB Simscape Multibody.
4) Task-space trajectory planning with square (pick-and-place) and circular (stirring) paths.
5) Experimental validation on physical hardware through a tea-preparation demonstration.

II. System Design

A. Mechanical Architecture

The manipulator possesses 4 Degrees of Freedom (DOF) and is constructed primarily from 3D-printed PLA parts, ensuring a lightweight yet rigid structure (Fig. 1). The arm is actuated by six servo motors: three high-torque MG996R

(180°, metal gear) servos for the base, shoulder, and elbow joints, and three SG90 micro servos (180°, plastic gear) for the wrist and gripper. One SG90 servo serves as a fixed dummy pivot to maintain the 4-DOF kinematic chain. Table I summarizes the primary components.

TABLE I: Hardware Components

Component	Qty
3D Printed Parts (PLA)	1 set
MG996R Servo Motors	3
SG90 Micro Servos	3
Arduino Uno	1
Power Supply (12V)	1

B. End-Effector Design

A parallel gripper serves as the primary end-effector, providing sufficient range and gripping force for lightweight objects such as cups and utensils. The inner surfaces of the jaws are lined with high-friction material to ensure stable grasping. The same gripper can hold a teaspoon attachment for instant beverage preparation, dispensing coffee powder or sugar before stirring—preserving modularity without additional tool-changing mechanisms.

C. Electronics and Communication

The system is powered by a 12V DC supply with a DC-DC step-down converter for voltage regulation. Control logic is executed on an Arduino Uno, which generates PWM signals to control servo positions. Communication between the PC-based trajectory planner and the Arduino operates at 115,200 baud using comma-separated joint angle strings (e.g., `"90,45,-30,0"`) with dedicated gripper commands (`GRIPPER_OPEN`, `GRIPPER_CLOSE`). A handshake protocol using a `"READY"` acknowledgment signal ensures motion synchronization. Table II summarizes the pin assignments.

TABLE II: Arduino Pin Assignments and Calibration

Joint	Pin	Calibration
Base (J1)	3	$q + 90°$
Shoulder (J2)	5	$-q + 110°$
Elbow (J3)	6	$-q + 90°$
Wrist (J4)	9	$-q + 110°$
Gripper	10	$q + 90°$ (clamped)

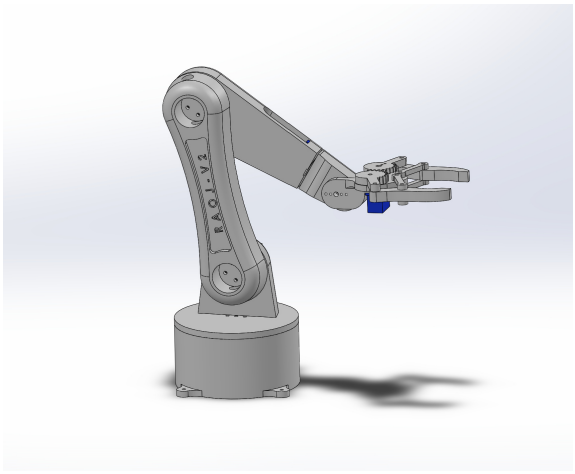


Fig. 1: CAD model of the 4-DOF robotic manipulator showing the joint arrangement and structural proportions.

III. Kinematic Modeling

A. Coordinate Frames and DH Parameters

Coordinate frames were assigned following the Denavit–Hartenberg convention [7], [8] using the X–Z rule (Fig. 2). The axes x_1, x_2, and x_3 lie in the same plane, while z_1 and z_4 are parallel. The z_3 axis is oriented opposite to z_1 and z_4, consistent with the DH convention for this manipulator. The resulting DH parameters are listed in Table III.

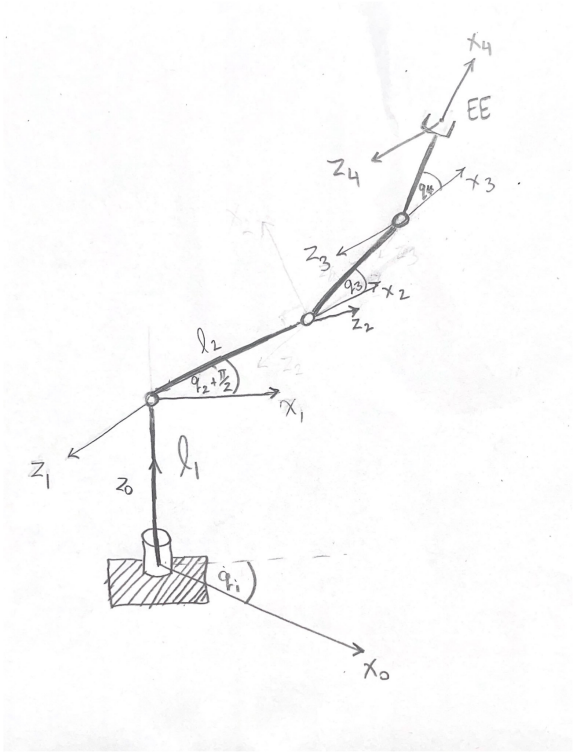


Fig. 2: Denavit–Hartenberg coordinate frame assignment for the 4-DOF manipulator.

TABLE III: Denavit–Hartenberg Parameters

Link	θ_i	d_i (m)	a_i (m)	α_i (rad)
1	q_1	0.04355	0	$\pi/2$
2	$q_2 + \pi/2$	0	0.140	π
3	q_3	0	0.133	π
4	q_4	0	0.109	0

B. Forward Kinematics

The forward kinematics function $F(\mathbf{q})$ computes the end-effector position $\mathbf{X} = [x, y, z]^T$ from the joint vector $\mathbf{q} = [q_1, q_2, q_3, q_4]^T$. Each link transformation is computed using the standard DH matrix:

$$ {}^{i-1}T_i = \begin{bmatrix} c_{\theta_i} & -s_{\theta_i}c_{\alpha_i} & s_{\theta_i}s_{\alpha_i} & a_i c_{\theta_i} \\ s_{\theta_i} & c_{\theta_i}c_{\alpha_i} & -c_{\theta_i}s_{\alpha_i} & a_i s_{\theta_i} \\ 0 & s_{\alpha_i} & c_{\alpha_i} & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{1} $$

where $c_{\theta_i} = \cos\theta_i$ and $s_{\theta_i} = \sin\theta_i$. The global transformation is obtained as ${}^0T_4 = {}^0T_1 \cdot {}^1T_2 \cdot {}^2T_3 \cdot {}^3T_4$. For the zero joint configuration ($q_1 = q_2 = q_3 = q_4 = 0$), the end-effector is elevated to $z = 0.42555\,\text{m}$ along the base z-axis, which equals $d_1 + a_2 + a_3 + a_4 = 0.04355 + 0.140 + 0.133 + 0.109\,\text{m}$.

The FK model was implemented identically in both MATLAB (`forward_kinematics_func.m`) and Python (ROS module `forward_kinematics.py`), ensuring consistent results across analytical computation and simulation.

C. Inverse Position Kinematics

Since the manipulator has 4 DOF but targets 3D Cartesian positions without full 6-DOF end-effector orientation constraints, the position kinematics problem is solved numerically using Newton–Raphson iterations over the 3×4 linear Jacobian. The position error is defined as:

$$ \mathbf{e}_k = \mathbf{X}_{\text{des}} - F(\mathbf{q}_k) \tag{2} $$

In the ROS/Python implementation, the joint angle update uses a Damped Least-Squares (DLS) formulation:

$$ \Delta\mathbf{q} = J_v^T(\mathbf{q}) \left[J_v(\mathbf{q}) J_v^T(\mathbf{q}) + \lambda^2 I_3 \right]^{-1} \mathbf{e} \tag{3} $$

where $J_v(\mathbf{q}) \in \mathbb{R}^{3\times4}$ is the analytical position Jacobian and $\lambda = 0.01$ is the damping factor for stability near singularities (whereas the MATLAB solver uses the undamped Moore–Penrose pseudoinverse $J_v^+(\mathbf{q})\mathbf{e}$).

D. Analytical Jacobian

The 3×4 position Jacobian $J_v(\mathbf{q}) = \partial F / \partial \mathbf{q}$ was derived analytically from $F(\mathbf{q}) = [-c_1 R,\ -s_1 R,\ d_1 + l_2 c_2 + l_3 c_\delta + l_4 c_\sigma]^T$, where $\delta = q_2 - q_3$, $\sigma = q_2 - q_3 + q_4$, $s_1 = \sin q_1$, $c_1 = \cos q_1$, $R = l_2 s_2 + l_3 s_\delta + l_4 s_\sigma$, $C = l_2 c_2 + l_3 c_\delta + l_4 c_\sigma$, $R' = l_3 s_\delta + l_4 s_\sigma$, and $C' = l_3 c_\delta + l_4 c_\sigma$:

$$ J_v = \begin{bmatrix} s_1 R & -c_1 C & c_1 C' & -l_4 c_\sigma c_1 \\ -c_1 R & -s_1 C & s_1 C' & -l_4 c_\sigma s_1 \\ 0 & -R & R' & -l_4 s_\sigma \end{bmatrix} \tag{4} $$

This analytical Jacobian matches central finite differences of $F(\mathbf{q})$ to within 10^{-9} across random workspace configurations.

E. Velocity Kinematics

Forward velocity kinematics relates joint velocities $\dot{\mathbf{q}}$ to the end-effector twist $\mathbf{V} = [v_x, v_y, v_z, \omega_x, \omega_y, \omega_z]^T$ via the 6×4 geometric Jacobian:

$$ \mathbf{V} = J(\mathbf{q}) \cdot \dot{\mathbf{q}} = \begin{bmatrix} J_v(\mathbf{q}) \\ J_\omega(\mathbf{q}) \end{bmatrix} \dot{\mathbf{q}} \tag{5} $$

The inverse velocity kinematics recovers joint velocities from a desired twist using the pseudo-inverse:

$$ \dot{\mathbf{q}} = J^+(\mathbf{q}) \cdot \mathbf{V}_{\text{des}} \tag{6} $$

Validation was performed by commanding joint velocities in Gazebo, computing the theoretical end-effector velocity $\mathbf{V}_{\text{theory}} = J(\mathbf{q})\dot{\mathbf{q}}$, and comparing it against the numerical velocity obtained from finite-difference position measurements. Both values showed close agreement, confirming the correctness of the Jacobian.

IV. Trajectory Planning

The trajectory planner generates a sequence of end-effector positions $\mathbf{X}(t) = [x(t), y(t), z(t)]^T$ sampled at $T_s = 0.1\,\text{s}$. The IK solver converts these into joint angle sequences $\mathbf{q}(t)$.

A. Square Trajectory (Pick-and-Place)

The square trajectory is defined in the Y–Z plane as a three-sided "gate-like" path connecting four waypoints for pick-and-place operations. The motion sequence consists of: (1) vertical ascent from the pickup location P_1 to a safe height P_2; (2) horizontal translation at the safe height to the drop-off column P_3; and (3) vertical descent to the placement point P_4. Position is linearly interpolated between consecutive waypoints:

$$ \mathbf{P}(t) = \mathbf{P}_{\text{start}} + \frac{t}{T_f}(\mathbf{P}_{\text{end}} - \mathbf{P}_{\text{start}}) \tag{7} $$

where $T_f = 2.0\,\text{s}$ per segment in MATLAB (3.0 s in the ROS trajectory node).

B. Circular Trajectory (Stirring)

The circular trajectory is defined in the X–Y plane for stirring operations. Given center $\mathbf{C} = [c_x, c_y, c_z]$ and radius r:

$$ \theta(t) = \frac{2\pi}{T_{\text{total}}} \cdot t \tag{8} $$

$$ x(t) = c_x + r\cos\theta(t) \tag{9} $$

$$ y(t) = c_y + r\sin\theta(t) \tag{10} $$

$$ z(t) = c_z \quad \text{(constant)} \tag{11} $$

This ensures smooth continuous motion with constant angular velocity $\omega = 2\pi/T_{\text{total}}$, where $T_{\text{total}} = 10\,\text{s}$. Table IV summarizes the key planning parameters.

TABLE IV: Trajectory and IK Solver Parameters

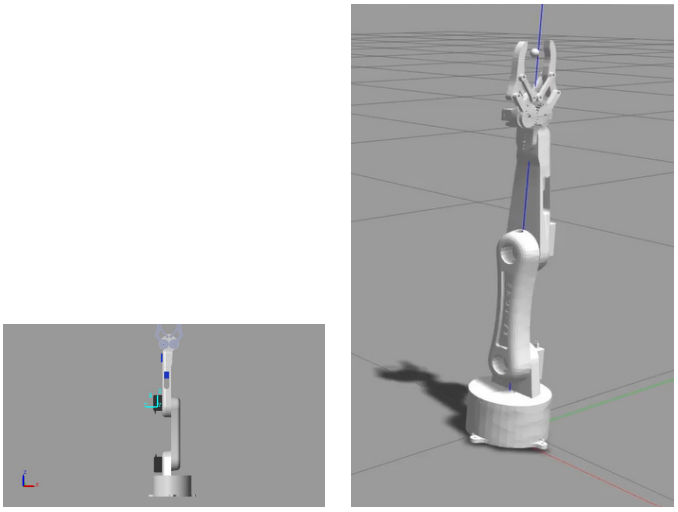
Parameter	Value
Sampling time T_s	0.1 s
Square segment duration T_f	2.0 s (MATLAB) / 3.0 s (ROS)
Circle total duration T_{total}	10 s
Damping factor λ	0.01 (ROS DLS) / 0 (MATLAB `pinv`)
IK max iterations	100 (MATLAB) / 500 (ROS)
Position tolerance $\|\mathbf{e}\|$	10^{-3} m (MATLAB) / 10^{-4} m (ROS)

V. Simulation and Validation

A. Dual-Platform Simulation

The kinematic model and trajectories were validated in two simulation environments: Gazebo (physics-based, ROS-integrated) and Simscape Multibody (system-level, MATLAB/Simulink). In Gazebo, Python ROS nodes published joint commands to the robot URDF model and queried the end-effector link pose via the `/gazebo/get_link_state` service (`my_robot::EE_frame` at 30 Hz, republished to `/end_effector_pose` by `ee_pose_from_gazebo.py`). In Simscape, the CAD assembly was imported into Simulink with MATLAB Function blocks computing FK for direct comparison.

Both environments produced end-effector positions matching the analytical model for all tested configurations. Fig. 3 shows the robot models in both simulation environments.



(a) Simscape Multibody (b) Gazebo

Fig. 3: Robot models in both simulation environments.

B. Forward Kinematics Verification

Table V presents the FK verification results verified in both GNU Octave/MATLAB and Python, showing end-effector positions and velocities for two test configurations. The zero-configuration case yields the exact theoretical sum $z = 0.42555$ m (0.426 m rounded), confirming the DH parameter chain.

TABLE V: Kinematic Benchmark Test Cases

Case	Input q (rad)	Output x (m)	Output V
1	$[0, 0, 0, 0]$	$[0, 0, 0.42555]$	$[0, 0, 0, 0, 0, 0.1]$
2	$[0.2, 0.4, -0.3, 0.1]$	$[-0.214, -0.043, 0.350]$	$[-0.081, \ldots]$

C. Inverse Kinematics Roundtrip Validation

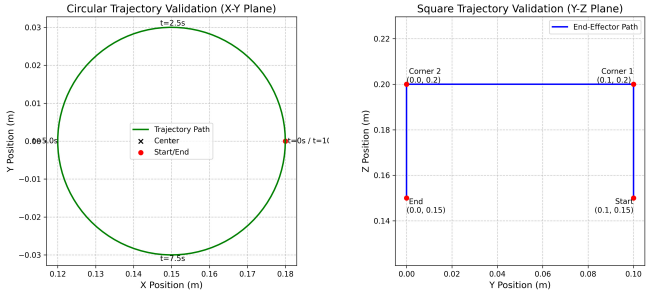
Three roundtrip tests were executed across both the Python solver (`validate_roundtrip.py`) and the Octave/MATLAB suite (`test_velocity_kinematics.m`), summarized in Table VI. Starting from $\mathbf{q}_0 = \mathbf{0}$ (cold start), the position IK solver converges to within 5.66×10^{-5} m in Python (under 10^{-4} m tolerance) and 4.96×10^{-4} m in Octave/MATLAB. When warm-started at the target pose, $\|\mathbf{q}_{\text{rec}} - \mathbf{q}\|$ is zero to machine precision ($\leq 1.39 \times 10^{-17}$ rad). For velocity recovery, undamped Moore–Penrose `pinv` reproduces $\dot{\mathbf{q}}$ to 10^{-15} rad/s, whereas DLS regularization ($\lambda = 0.01$) introduces bounded bias up to 8.42×10^{-3} rad/s in exchange for numerical stability near singularities.

TABLE VI: Python and Octave Roundtrip Validation

Case	Input q (rad)	$\|\mathbf{e}_{\text{cold}}\|$ (m)	$\|\Delta\dot{\mathbf{q}}_{\text{DLS}}\|$	$\|\Delta\dot{\mathbf{q}}_{\text{pinv}}\|$
1	$[0, 0, 0, 0]$	0.00	1.00×10^{-5}	0.00
2	$[0.2, 0.4, -0.3, 0.1]$	5.66×10^{-5}	5.00×10^{-3}	7.26×10^{-16}
3	$[-0.3, 0.6, 0.2, -0.2]$	5.45×10^{-5}	8.42×10^{-3}	1.10×10^{-15}

D. Trajectory Validation

Both square and circular trajectories were executed in Gazebo and Simscape, with joint-angle sequences generated by the custom Python planner. Fig. 4 shows the trajectory validation results. For the square path, the four corners were traced with high fidelity and distinct transitions between linear segments. For the circular path, the end-effector maintained constant z-height while tracing a nearly perfect circle in the X–Y plane, with small deviations due to discretization and numerical IK residuals. The end-effector traces from Simscape closely matched Gazebo results, confirming cross-platform consistency despite differences in numerical integrators.



(a) Circular trajectory (b) Square trajectory

Fig. 4: Task-space trajectory validation results from simulation.

VI. Hardware Implementation

A. Assembly and Wiring

The physical manipulator was fully assembled from 3D-printed PLA parts with all servo motors mounted at their respective joints (Fig. 6). The robot's base is secured to a weighted wooden board to prevent tipping during high-speed movements. All electronics, including the Arduino Uno and motor connections, are housed in a protective enclosure. The circuit schematic (Fig. 5) shows the six servo motors connected to the Arduino's PWM pins, with the 12V external power regulated through a DC-DC converter.

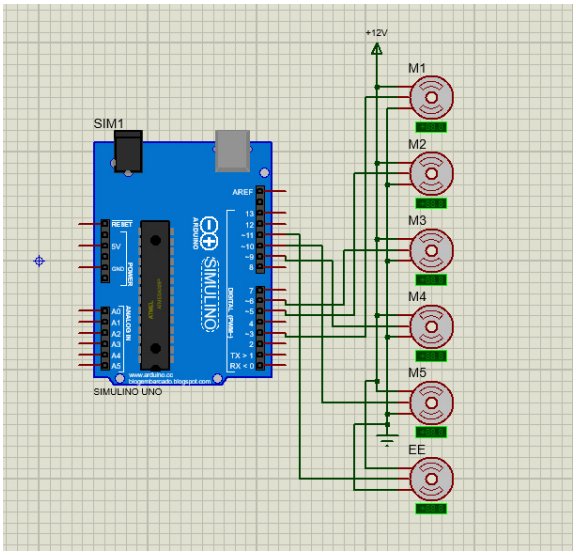


Fig. 5: Circuit schematic: Arduino Uno connected to six servo motors (M1–M5 and EE) via PWM pins, with 12V external power and common ground.

B. Open-Loop Validation

Initial hardware testing used Arduino scripts to sequentially actuate each motor through its full range of motion: base rotation, shoulder link, elbow link, wrist rotation, and gripper actuation. The fixed servo (motor 4) was confirmed stationary, consistent with the 4-DOF design. All active degrees of freedom responded correctly to commands, validating hardware readiness for trajectory execution.

C. Trajectory Execution

The joint-angle sequences computed by the trajectory planner were exported as CSV files and transmitted to the Arduino via a dedicated Python script (`send_trajectory.py`) that reads files line-by-line and waits for `"READY"` acknowledgments. The circular trajectory produced smooth, continuous motor movement, validating the 0.1 s time-step resolution. The square trajectory corners were reached with reasonable accuracy, with minor deviations attributed to mechanical backlash in servo gears and arm deflection under gravity at full extension.



Fig. 6: Fully assembled 4-DOF robotic manipulator in the experimental setup.

VII. Application: Tea Preparation

The tea-preparation scenario serves as the primary validation task, demonstrating coordinated task-space motion and gripper actuation in a realistic service context. The complete task consists of five motion phases executed sequentially:

1) **Sugar Transport:** The manipulator executes the first square trajectory to pick up a sugar cube from its container, transport it, and release it into the cup.

2) **Spoon Approach:** A second square trajectory moves the end-effector to the spoon location and grasps the spoon handle.
3) **Spoon Transport:** The arm lifts the spoon and transports it toward the cup using a reverse square path.
4) **Linear Centering:** A short linear translation centers the spoon within the cup frame before stirring.
5) **Stirring:** The robot performs the circular trajectory for three full cycles ($3 \times 10\,\mathrm{s}$), maintaining the spoon vertically inside the cup.

The complete 3D trajectory combining all phases is visualized in Fig. 7, illustrating the spatial relationship between the task objects (sugar container, spoon, cup) and the robot's path.

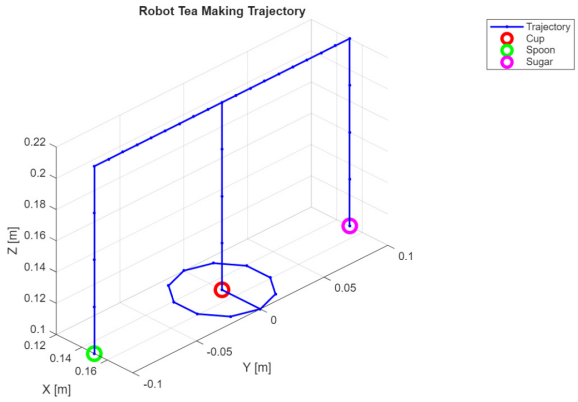


Fig. 7: 3D visualization of the complete tea-preparation trajectory, showing waypoints for sugar transport, spoon approach, and stirring.

Qualitatively, the hardware execution matched the simulation behavior. The square corners were clearly visible in the arm motion, and the circular stirring appeared smooth and repeatable. Small deviations from ideal paths arise from servo backlash, link flexibility, open-loop control, and slight misalignments between modeled and actual object positions. Despite these imperfections, the system successfully demonstrated coordinated multi-step manipulation in a practical service task.

VIII. Discussion

The experimental results confirm that a low-cost, 3D-printed 4-DOF manipulator can perform multi-step beverage preparation tasks with acceptable repeatability for tabletop demonstrations. The DH-based kinematic model achieves sub-millimeter cold-start position convergence ($< 10^{-4}\,\mathrm{m}$ in ROS Python) and machine-precision velocity roundtrip residuals ($\leq 2.2 \times 10^{-15}\,\mathrm{rad/s}$ under `pinv`), demonstrating that the analytical formulation is mathematically consistent across Octave/MATLAB and Python.

Two structural limitations shape the current platform. First, because the arm has 4 DOF operating in 3D workspace coordinates, position IK leaves end-effector orientation unconstrained rather than controlling full 6-DOF pose. Second, the physical hardware executes open-loop servo commands without encoder feedback, making joint angles susceptible to gear backlash and structural deflection under gravity, particularly at abrupt square-trajectory corners; the smoother circular trajectory exhibits noticeably less chatter.

The use of locally sourced components and 3D printing eliminates specialized industrial hardware, keeping the platform accessible for educational robotics laboratories.

IX. Conclusion

This paper presented the development of a 4-DOF service robotic manipulator, covering mechanical CAD design, DH kinematic modeling, ROS/Gazebo and Simscape simulation, and Arduino hardware execution. The analytical forward kinematics and Jacobian agree with finite-difference checks to 10^{-9}, cold-start position IK converges below $10^{-4}\,\mathrm{m}$, and undamped velocity roundtrip tests achieve $10^{-15}\,\mathrm{rad/s}$ precision. Task-space square and circular trajectories were verified in simulation and demonstrated physically through a five-phase tea-preparation task.

Future work will focus on closed-loop joint or vision feedback to compensate for servo backlash and gravity droop, along with wrist-pitch orientation constraints during liquid handling.

References

[1] S. Park, M. K. Park, J. Heo, J. S. Hwang, S. Hwang, D. Kim, S.-J. Chung, and H. S. Kwak, "Robot versus human barista: Comparison of volatile compounds and consumers' acceptance, sensory profile, and emotional response of brewed coffee," *Food Research International*, vol. 172, p. 113119, 2023.

[2] A. Setiawan and A. Ma'arif, "Stirring system design for automatic coffee maker using OMRON PLC and PID control," *Int. J. Robotics and Control Systems*, vol. 1, no. 3, pp. 390–401, 2021.

[3] S. Michalková, K. Wojnarowský, Š. Kolenyák, M. Kantor, and J. Arm, "Automating the handling process of a coffee machine using a robotic manipulator," in *Proc. XV Int. Symp. Industrial Electronics and Applications (INDEL)*, 2024, pp. 1–5.

[4] H. P. Nurba, D. Hadian, N. Lestari, K. A. Munastha, H. Mistialustina, and E. Rachmawati, "Performance evaluation of 3 DOF arm robot with forward kinematics Denavit–Hartenberg method for coffee maker machine," in *Proc. IEEE Int. Conf. Telecommunication Systems, Services, and Applications (TSSA)*, 2022, pp. 1–6.

[5] H. D. Salman, M. N. Hamzah, and S. H. Bakhy, "Kinematics analysis and implementation of three degrees of freedom robotic arm by using MATLAB," *The Iraqi Journal for Mechanical and Materials Engineering*, vol. 21, no. 2, pp. 118–129, 2021.

[6] A. Waluyo, A. Tafrikhatin, and S. R. Heri, "Robot arm design for coffee maker Arduino based," in *Proc. 2nd Borobudur Int. Symposium on Science and Technology (BIS-STE 2020)*, Atlantis Press, 2021, pp. 438–442.

[7] J. J. Craig, *Introduction to Robotics: Mechanics and Control*, 3rd ed. Pearson Prentice Hall, 2005.

[8] B. Siciliano, L. Sciavicco, L. Villani, and G. Oriolo, *Robotics: Modelling, Planning and Control*. Springer, 2009.